3-Tier Architecture Using Terraform and AWS Application Load Balancer

Project Name: 3-Tier Architecture with Application Load Balancer Using Terraform

Created By: Meduri Krishna Teja

Project Overview

This project demonstrates how to build a complete 3-tier architecture on AWS using Terraform. The infrastructure consists of separate Web, Application, and Database layers deployed on EC2 instances. AWS Application Load Balancers are used to distribute incoming traffic to the Web and Application servers.

Architecture Components

- **Web Layer:** Nginx running on an EC2 instance
- **Application Layer:** Apache Tomcat running on an EC2 instance
- **Database Layer:** MySQL running on an EC2 instance
- **Networking:** Custom VPC with three subnets across different Availability Zones
- **Load Balancing:** Application Load Balancers for Nginx and Tomcat

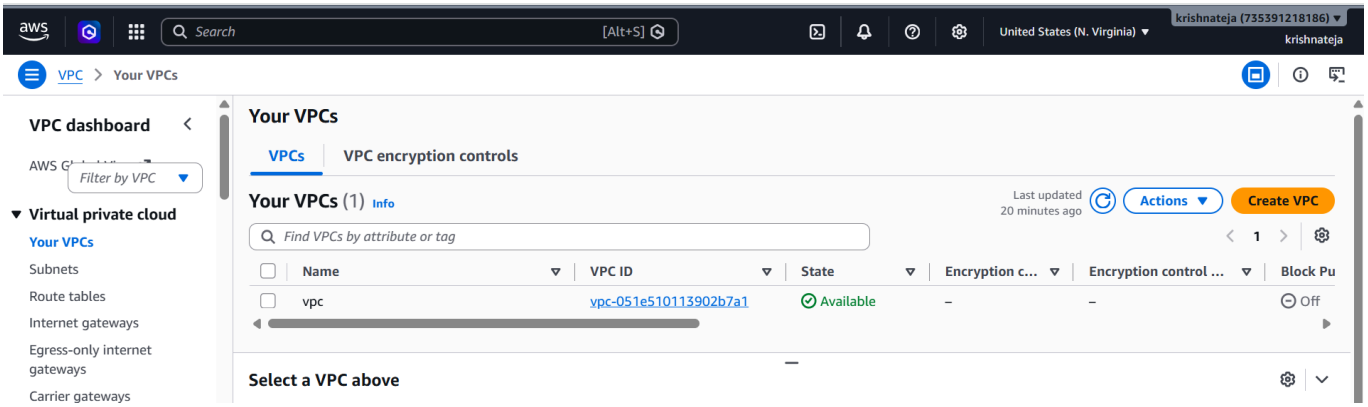
Steps to create a 3-Tier Architecture:
Step1: Configure the AWS Provider
The project begins by configuring the AWS provider in Terraform. The deployment region used is US East (N. Virginia).

```
provider "aws"{
    access_key = "AKIA2WOF3PIFLV7VSLEJ"
    secret_key = "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"
    region = "us-east-1"
}
```

Step2: Create the VPC
A Virtual Private Cloud (VPC) is created with a custom CIDR block. AWS automatically creates the default Route Table, Network ACL, and Security Group for the VPC.

```
resource "aws_vpc" "vpc" {
  cidr_block = var.vpcip
  tags = {
    Name = var.vpcname
  }
}
```

Result:
Custom VPC Created Successfully.



Step3: Create Three Subnets

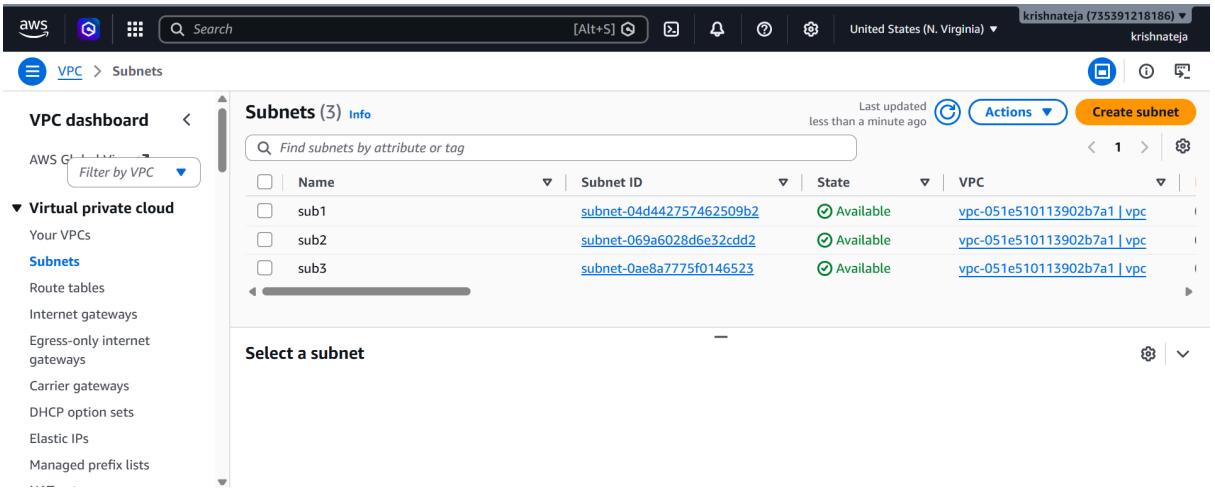
Three public subnets are created across different Availability Zones to improve availability and fault tolerance.

- Subnet 1 – Web Server
- Subnet 2 – Application Server
- Subnet 3 – Database Server

```
resource "aws_subnet" "sub1" {
    vpc_id = aws_vpc.vpc.id
    cidr_block = var.subnet1ip
    availability_zone = "us-east-1a"
    tags = {
        Name = var.sub1name
    }

}

resource "aws_subnet" "sub2" {
    vpc_id = aws_vpc.vpc.id
    cidr_block = var.subnet2ip
    availability_zone = "us-east-1b"
    tags = {
        Name = var.sub2name
    }

}

resource "aws_subnet" "sub3" {
    vpc_id = aws_vpc.vpc.id
    cidr_block = var.subnet3ip
    availability_zone = "us-east-1c"
    tags = {
        Name = var.sub3name
    }

}
```

Result:

Three subnets were successfully deployed across multiple Availability Zones.



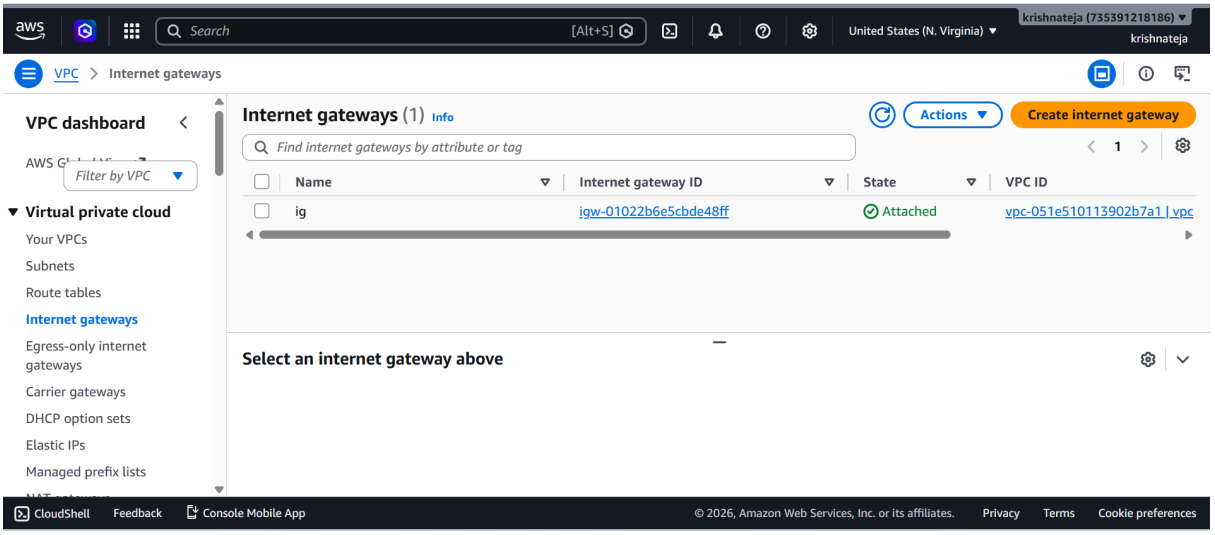
Step4: Create and Attach an Internet Gateway

An Internet Gateway is created and attached to the VPC, enabling internet access for resources deployed in public subnets.

```
resource "aws_internet_gateway" "ig" {
  vpc_id = aws_vpc.vpc.id
  tags = {
    Name = var.igname
  }
}
```

Result:

Internet Gateway attached successfully.



Step5: Configure the Route Table

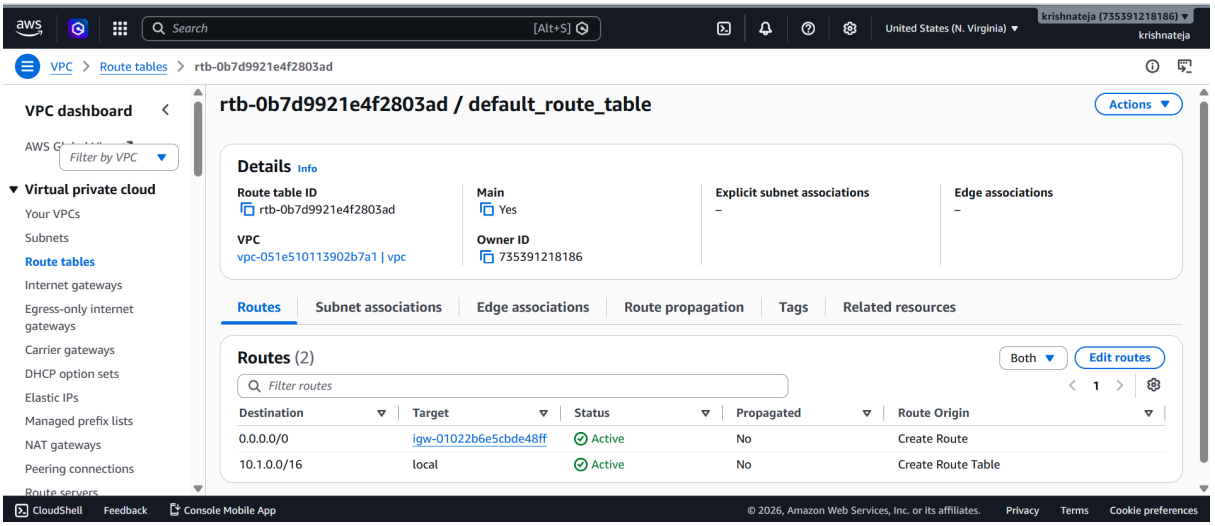
The default Route Table is updated with a route (`0.0.0.0/0`) pointing to the Internet Gateway, allowing outbound internet connectivity.

```
resource "aws_default_route_table" "drt" {
  default_route_table_id = aws_vpc.vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
```

```
    gateway_id = aws_internet_gateway.ig.id
  }
  tags = {
    Name = var.drtname
  }
}
```

Result:

Public internet access is configured successfully.



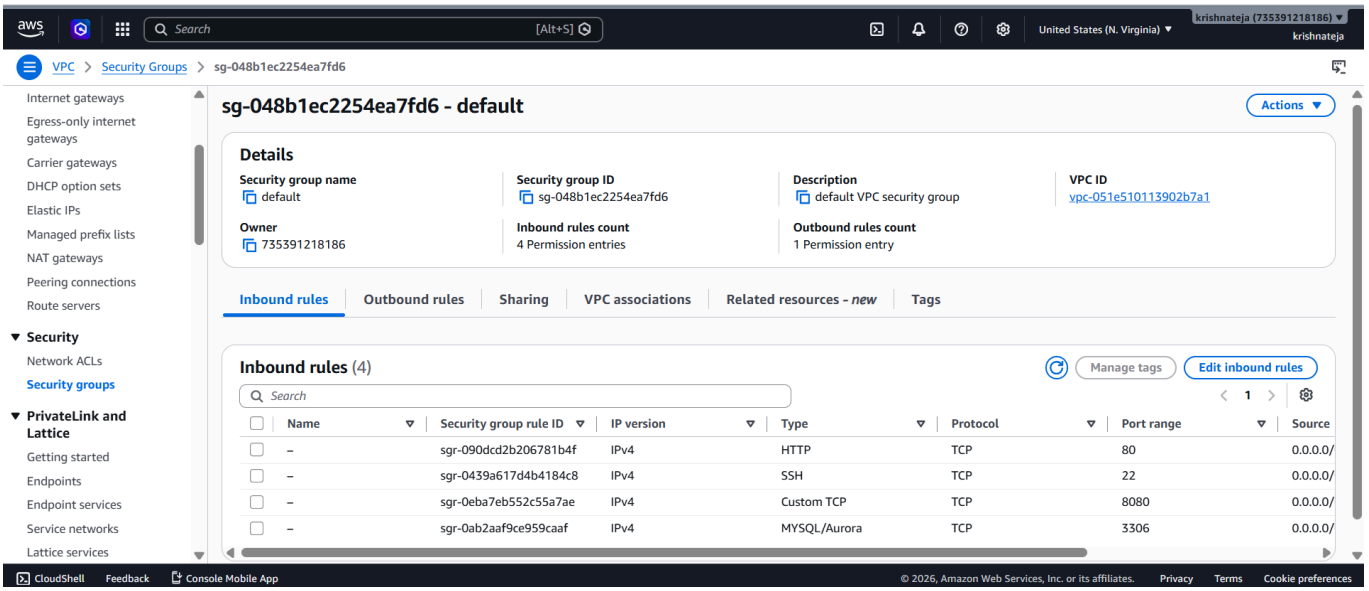
Step6: Configure the Security Group

A Security Group is configured with the required inbound rules.

```
resource "aws_default_security_group" "dsg" {
  vpc_id = aws_vpc.vpc.id

   ingress {
    description = "SSH from anywhere"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # For production, restrict this to your specific
local IP
  }
  ingress {
    description = "Allow public HTTP traffic"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    description = "Allow public Custom TCP"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }
   ingress {
    description = "Allow public MYSQL/Aurora"
    from_port   = 3306
    to_port     = 3306
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }


  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name=var.dsgname
  }
}
```

Result:



Step7: Launch an EC2 Instance

Three EC2 instances are provisioned using Terraform.

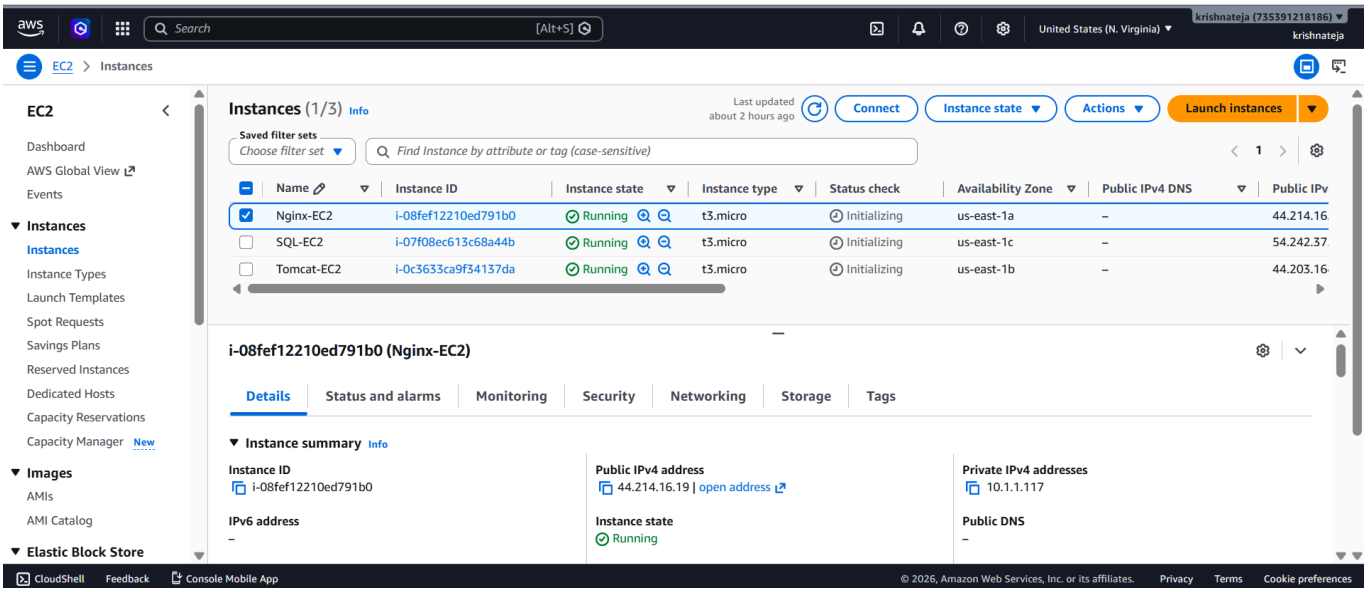
Instances	**Purpose**
Nginx EC2	Web Server
Tomcat EC2	Application Server
MySQL EC2	Database Server

```
resource "aws_instance" "EC2_1" {
  ami = var.aminame
  instance_type = var.instancetype
  subnet_id = aws_subnet.sub1.id
  key_name = "virginia"
  associate_public_ip_address = true
```

```
  tags = {
    Name = var.ec2_1name
  }
}

resource "aws_instance" "EC2_2" {
  ami = var.aminame
  instance_type = var.instancetype
  subnet_id = aws_subnet.sub2.id
  key_name = "virginia"
  associate_public_ip_address = true
  tags = {
    Name = var.ec2_2name
  }
}

resource "aws_instance" "EC2_3" {
  ami = var.aminame
  instance_type = var.instancetype
  subnet_id = aws_subnet.sub3.id
  key_name = "virginia"
  associate_public_ip_address = true
  tags = {
    Name = var.ec2_3name
  }
}
```

Result:

All three EC2 instances launched successfully.



Step8: Install and Configure Nginx

Connect to the web server with MobaXterm

root@ip-10-1-1-219:/home/ubuntu#

cmd: apt update

cmd: apt install nginx -y

```
Selecting previously unselected package nginx-common.
(Reading database ... 85303 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1.6_all.deb ...
Unpacking nginx-common (1.28.3-2ubuntu1.6) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1.6_amd64v3.deb ...
Unpacking nginx (1.28.3-2ubuntu1.6) ...
Setting up nginx-common (1.28.3-2ubuntu1.6) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/usr/lib/systemd/system/nginx.service'.
Setting up nginx (1.28.3-2ubuntu1.6) ...
 * Upgrading binary nginx                                                                                              [
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for ufw (0.36.2-9build1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

Now we check if Nginx is installed on our web server.

Nginx was installed successfully on our web server

Step9: Install Apache Tomcat

Connect to the application server with MobaXterm

cmd: sudo apt update

cmd: apt install default-jdk -y

cmd: wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.49/bin/apache-tomcat- 11.0.14.tar.gz

cmd: ls -lt

cmd: tar -xvzf apache-tomcat-11.0.14.tar.gz

cmd: ls -lt

cmd: mv apache-tomcat-11.0.14 tomcat

cmd: ls

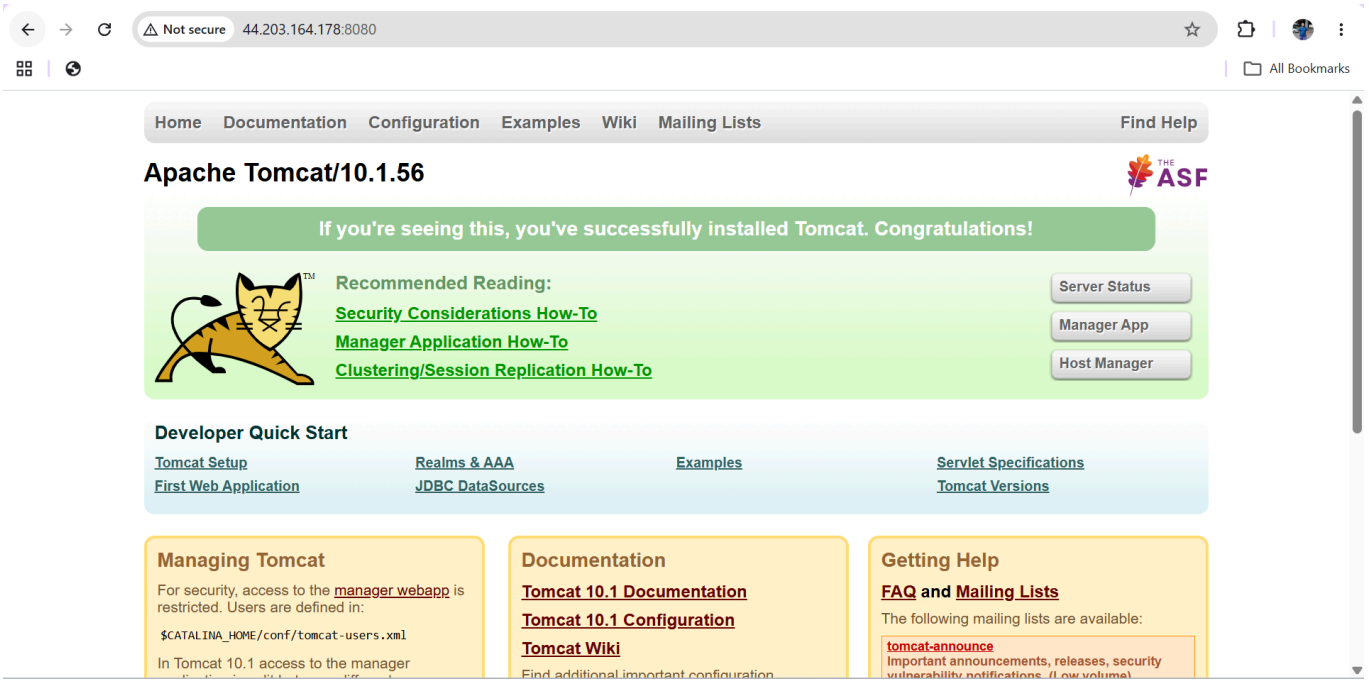
cmd: cd tomcat/bin/ or cd /home/ubuntu/tomcat/bin

cmd: ls

cmd: ./startup.sh

```
apache-tomcat-10.1.56/bin/tool-wrapper.sh
apache-tomcat-10.1.56/bin/version.sh
root@ip-10-1-2-238:/home/ubuntu# ls -lt
total 14060
drwxr-xr-x 9 root root     4096 Jul  7 05:45 apache-tomcat-10.1.56
-rw-r--r-- 1 root root 14389873 Jun 17 23:40 apache-tomcat-10.1.56.tar.gz
root@ip-10-1-2-238:/home/ubuntu# mv apache-tomcat-10.1.56 tomcat
root@ip-10-1-2-238:/home/ubuntu# ls
apache-tomcat-10.1.56.tar.gz  tomcat
root@ip-10-1-2-238:/home/ubuntu# cd tomcat/bin/
root@ip-10-1-2-238:/home/ubuntu/tomcat/bin# ls
bootstrap.jar       ciphers.sh                  daemon.sh     migrate.bat       shutdown.sh          tool-wrapper.bat
catalina-tasks.xml  commons-daemon-native.tar.gz  digest.bat    migrate.sh        startup.bat          tool-wrapper.sh
catalina.bat        commons-daemon.jar          digest.sh     setclasspath.bat  startup.sh           version.bat
catalina.sh         configtest.bat              makebase.bat  setclasspath.sh   tomcat-juli.jar      version.sh
ciphers.bat         configtest.sh               makebase.sh   shutdown.bat      tomcat-native.tar.gz
root@ip-10-1-2-238:/home/ubuntu/tomcat/bin# ./startup.sh
Using CATALINA_BASE:   /home/ubuntu/tomcat
Using CATALINA_HOME:   /home/ubuntu/tomcat
Using CATALINA_TMPDIR: /home/ubuntu/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /home/ubuntu/tomcat/bin/bootstrap.jar:/home/ubuntu/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
```

Now we check if Nginx is installed on our Application server.



Tomcat was installed successfully on our Application server.

Step10: Install MySQL

Connect to the db server with MobaXterm

Cmd: apt update

Cmd: apt install mysql-server –y

Cmd: mysql_secure_installation

➢ Edit the MySQL configuration file using a text editor:

vi /etc/mysql/mysql.conf.d/mysqld.cnf

➢ Locate the following line: bind-address = 127.0.0.1

• bind-address = 0.0.0.0

• Save and exit (CTRL + X, then Y, then Enter).

Cmd: systemctl restart mysql

Cmd: systemctl status mysql

Now we check if MySQL is installed on our DB server.

```
root@ip-10-1-3-176:/home/ubuntu# vi /etc/mysql/mysql.conf.d/mysqld.cnf
root@ip-10-1-3-176:/home/ubuntu# systemctl restart mysql
root@ip-10-1-3-176:/home/ubuntu# systemctl status mysql
● mysql.service - MySQL Community Server
     Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-07-07 05:58:23 UTC; 13s ago
 Invocation: bb300665a0564d228d2e999b63674e56
    Process: 6036 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
   Main PID: 6047 (mysqld)
     Status: "Server is operational"
      Tasks: 35 (limit: 627)
     Memory: 493.1M (peak: 493.2M)
        CPU: 856ms
     CGroup: /system.slice/mysql.service
             └─6047 /usr/sbin/mysqld

Jul 07 05:58:22 ip-10-1-3-176 systemd[1]: Starting mysql.service - MySQL Community Server...
Jul 07 05:58:23 ip-10-1-3-176 systemd[1]: Started mysql.service - MySQL Community Server.
```

MySQL was installed successfully on our DB server.

Step11: Verify Server Connectivity

Now connect the web server to the application server.

Open the web server in MobaXterm

Cmd: telnet 44.203.164.178 8080

In this, 44.203.164.178 is the application server's public IP. 8080 is the HTTP port number.

```
root@ip-10-1-1-117:/home/ubuntu# telnet 44.203.164.178 8080
Trying 44.203.164.178...
Connected to 44.203.164.178.
Escape character is '^]'.
Connection closed by foreign host.
root@ip-10-1-1-117:/home/ubuntu#
```

Our web server is successfully connected to the application server.

Step12: Verify Server Connectivity

Now connect the application server to the DB server.

Open the application server in MobaXterm

Cmd: telnet 54.242.37.142 3306

In this, 54.242.37.142 is the DB server's public IP. 3306 is the MySQL port number.

```
root@ip-10-1-2-238:/home/ubuntu/tomcat/bin# telnet 54.242.37.142 3306
Trying 54.242.37.142...
Connected to 54.242.37.142.
Escape character is '^]'.
cHost 'ec2-44-203-164-178.compute-1.amazonaws.com' is not allowed to connect to this MySQL serverConnection closed by foreign
host.
root@ip-10-1-2-238:/home/ubuntu/tomcat/bin#
```

Our Application server is successfully connected to the DB server.

Step13: Verify Server Connectivity

Now check whether the DB server is connected to the application server or not.

Open dbserver in MobaXterm

Cmd: telnet 44.203.164.178 8080

```
root@ip-10-1-3-176:/home/ubuntu# telnet 44.203.164.178 8080
Trying 44.203.164.178...
Connected to 44.203.164.178.
Escape character is '^]'.
Connection closed by foreign host.
root@ip-10-1-3-176:/home/ubuntu#
```

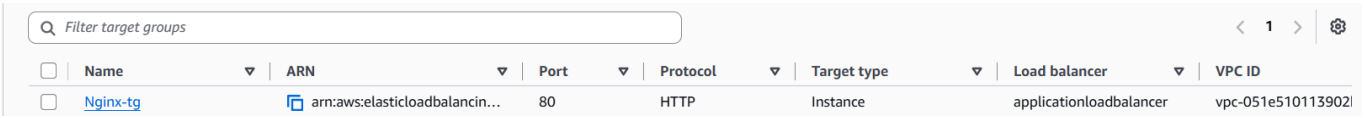
Our DB server is successfully connected to the Application server.

Step14: Create Target Group

Nginx Target Group (Port 80)

```
resource "aws_lb_target_group" "Nginx" {
  name     = "Nginx-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.vpc.id

  health_check {
    path = "/"
  }
}
```

The Output is:



Step15: Create Target Group

Tomcat Target Group (Port 8080)

```
resource "aws_lb_target_group" "Tomcat" {
  name     = "Tomcat-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.vpc.id

  health_check {
    path = "/"
  }
}
```

The Output is:

Step16: Create Application Load Balancer

Application Load Balancers are deployed across all three public subnets.

Each ALB is associated with the default Security Group and configured to forward traffic to its corresponding Target Group.

```
resource "aws_lb" "Nginx" {
  name               = "Nginx-Load-Balancer"
  internal           = false
  load_balancer_type = "application"

  security_groups = [aws_default_security_group.dsg.id]

  subnets = [
    aws_subnet.sub1.id,
    aws_subnet.sub2.id,
    aws_subnet.sub3.id
  ]
```

```
}
```

Result:

Nginx Application Load Balancer created.

Step17: **Create Application Load Balancer**

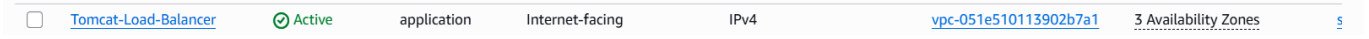
Application Load Balancers are deployed across all three public subnets.

Each ALB is associated with the default Security Group and configured to forward traffic to its corresponding Target Group.

```
resource "aws_lb" "Tomcat" {
  name               = "Tomcat-Load-Balancer"
  internal           = false
  load_balancer_type = "application"
  security_groups = [aws_default_security_group.dsg.id]
  subnets = [
    aws_subnet.sub1.id,
    aws_subnet.sub2.id,
    aws_subnet.sub3.id
  ]
}
```

Result:

Tomcat Application Load Balancer created.



Step18: Register EC2 Target

The Web Server EC2 instances are registered with their respective Target Group.

```
resource "aws_lb_target_group_attachment" "Nginx" {
  target_group_arn = aws_lb_target_group.Nginx.arn
  target_id        = aws_instance.EC2_1.id
  port             = 80
}
```

Step19: Register EC2 Target

The Application Server EC2 instances are registered with their respective Target Group.

```
resource "aws_lb_target_group_attachment" "Tomcat" {
  target_group_arn = aws_lb_target_group.Tomcat.arn
  target_id        = aws_instance.EC2_2.id
  port             = 8080
}
```

Step20: Configure Load Balancer Listeners

Create Listeners for the load balancer

```
resource "aws_lb_listener" "Nginx" {
  load_balancer_arn = aws_lb.Nginx.arn

  port     = 80
```

```
  protocol = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.Nginx.arn
  }
}
```

Step21: Configure Load Balancer Listeners

Create Listeners for the load balancer.

```
resource "aws_lb_listener" "Tomcat" {
  load_balancer_arn = aws_lb.Tomcat.arn

  port     = 8080
  protocol = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.Tomcat.arn
  }
}
```

Configure Variables:

```
variable "vpcip" {
  default = "10.1.0.0/16"
}
variable "vpcname" {
  default = "vpc"
}
variable "subnet1ip" {
  default = "10.1.1.0/24"
}
variable "sub1name"{
    default = "sub1"
}

variable "subnet2ip" {
  default = "10.1.2.0/24"
}
variable "sub2name"{
    default = "sub2"
}

variable "subnet3ip" {
  default = "10.1.3.0/24"
```

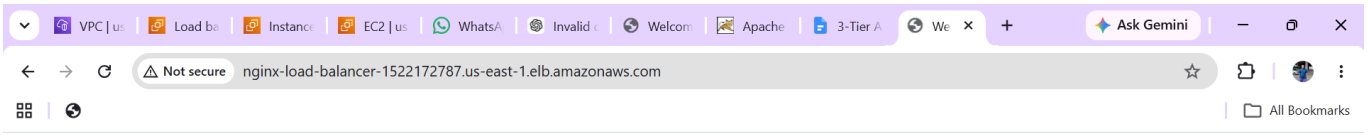
```
}
variable "sub3name"{
    default = "sub3"
}
variable "igname" {
  default = "ig"
}
variable "drtname" {
  default = "default_route_table"
}
variable "dsgname" {
  default = "default_security_group"
}
variable "aminame" {
  default = "ami-0b6d9d3d33ba97d99"
}
variable "instancetype" {
  default = "t3.micro"
}
variable "ec2_1name" {
  default = "Nginx-EC2"
}
variable "ec2_2name" {
  default = "Tomcat-EC2"
}
variable "ec2_3name" {
  default = "SQL-EC2"
}
```

Step22: Configure Outputs
Create the output of the Load Balancers' DNS Names

```
output "Nginx_DNS" {
  value = aws_lb.Nginx.dns_name
}
output "Tomcat_DNS" {
  value = aws_lb.Tomcat.dns_name
}
```

The Output of the Load Balancers' DNS names is
Outputs:

Nginx_DNS = "Nginx-Load-Balancer-1522172787.us-east-1.elb.amazonaws.com"
Tomcat_DNS = "Tomcat-Load-Balancer-2095237220.us-east-1.elb.amazonaws.com"
Copy and Paste the Nginx_DNS Name in the Web Browser.



Welcome to nginx!

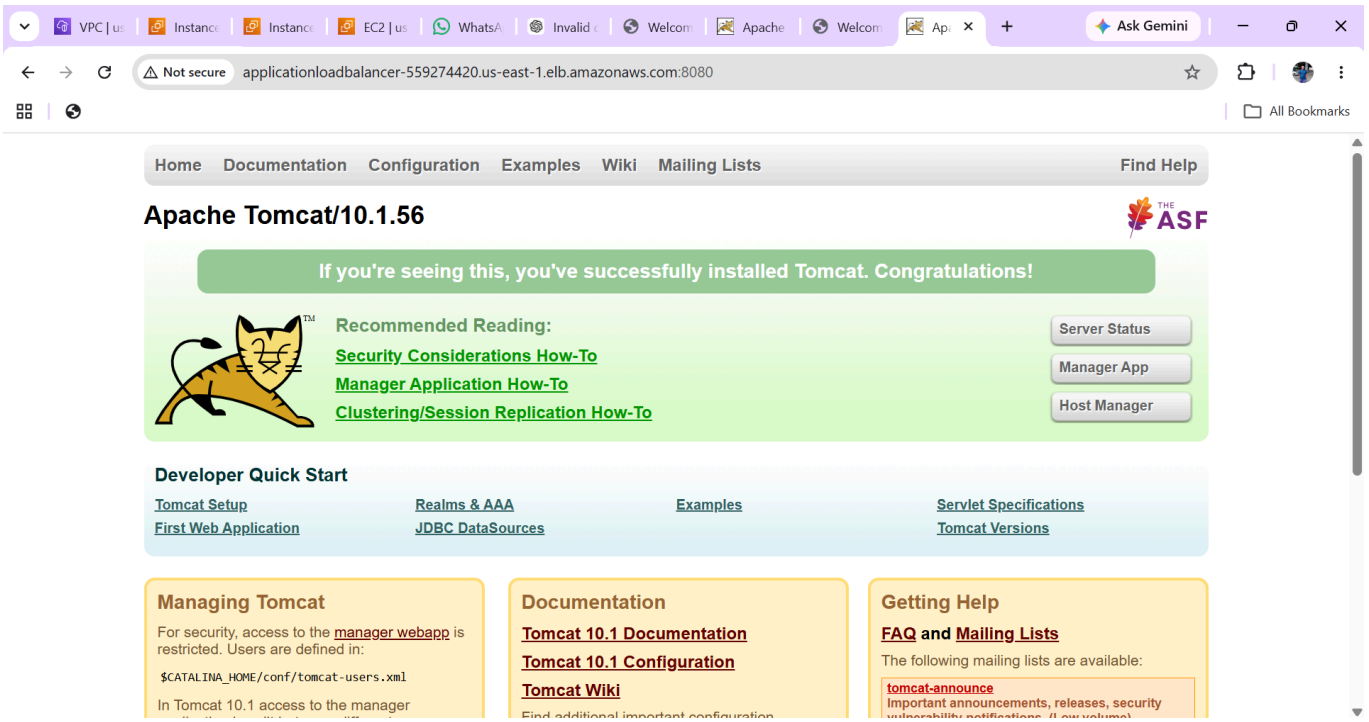
If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Copy and Paste the Tomcat_DNS Name in the Web Browser.

The Output is



Conclusion

This project successfully demonstrates the deployment of a scalable 3-tier architecture on AWS using Terraform. The infrastructure includes a custom VPC, public subnets, EC2 instances for the Web, Application, and Database tiers, Application Load Balancers, Target Groups, and Listeners. Infrastructure as Code (IaC) through Terraform ensures that the environment can be deployed consistently and efficiently with minimal manual effort.